

Shielded Q-learning for Thermally-Safe, Energy-Aware Multi-Model Intrusion Detection Scheduling on Lightweight Edge Devices

Nuonan Ouyang, Adrian Shatte, Zhigang Lu, Chao Chen, and Wei Xiang

Abstract—Lightweight edge intrusion detection must balance detection quality with strict latency, thermal, and energy budgets. We formulate multi-model intrusion-detection-system (IDS) scheduling as a constrained Markov decision process and propose Feasible-Set Shielded Q-learning (FSSQL-R): a calibrated runtime guard first restricts model selection to actions predicted to satisfy per-window latency and temperature caps, after which masked Q-learning selects within the admissible set. We evaluate the method through online measurements on Raspberry Pi 3B+ and Raspberry Pi 4B 8GB platforms, with separate device-specific calibration. Held-out guard coverage is 2879/2880 (0.999653) on the Pi 3B+ and 0.998958 joint coverage on the Pi 4B. Under a binding Pi 3B+ regime, the shield excludes LUCID but FSSQL-R does not outperform the simpler Safe-Greedy controller: mean F1 is 0.8523 versus 0.8708 and mean reward is 0.5434 versus 0.5839. In 60 Pi 4B online runs, FSSQL-R improves paired-seed F1 over Safe-Greedy by 0.114–0.121 and cumulative reward by 122–129, but Safe-Greedy is a degenerate reference in this experiment (FPR=1.0000 in every run). A no-training Threshold controller attains the highest mean F1 in all three Pi 4B conditions. Direct POWER-Z KM003C measurements quantify DC input energy, but do not establish a consistent FSSQL-R energy advantage. These results support calibrated feasible-set enforcement while showing that the incremental value of Q-learning is device- and regime-dependent rather than universal.

Index Terms—Lightweight IoT, intrusion detection, dynamic model scheduling, safe reinforcement learning, shielding, Q-learning, thermal safety, energy measurement, Raspberry Pi.

I. INTRODUCTION

Lightweight edge devices increasingly execute intrusion detection close to data sources, but deployability is constrained by latency deadlines, thermal headroom, and energy budgets. The central challenge is not only classification accuracy: a detector that is strong offline can become impractical when sustained execution increases latency, heats the device, or consumes excessive energy [1]–[3]. Maintaining a heterogeneous model pool and selecting among detectors online is therefore attractive, particularly when traffic risk and resource state vary over time.

N. Ouyang (corresponding author) and A. Shatte are with the College of Science and Engineering, James Cook University, Australia (e-mail: nuonan.ouyang@my.jcu.edu.au; adrian.shatte@jcu.edu.au).

Z. Lu is with the School of Computer, Data and Mathematical Sciences, Western Sydney University, Australia (e-mail: z.lu@westernsydney.edu.au).

C. Chen is with the School of Accounting, Information Systems & Supply Chain, RMIT University, Australia (e-mail: chao.chen@rmit.edu.au).

W. Xiang is with the Australian Centre for AI in Medical Innovation and the Cisco–La Trobe Centre for AI and Internet of Things, La Trobe University, Australia (e-mail: w.xiang@latrobe.edu.au).

Reinforcement learning (RL) provides a natural mechanism for long-horizon model selection, but unconstrained exploration can violate deployment limits. Constrained MDP approaches typically control expected cost [4]–[6], whereas embedded deadlines and temperature limits are operational constraints that may need runtime enforcement. Shielding addresses this by blocking actions that fail an admissibility test before the RL policy acts [7]–[9]. Recent IDS work has likewise combined a safety-aware RL policy with a runtime shield, although in a 5G self-healing remediation setting rather than resource-constrained detector scheduling [10].

This paper studies the deployment problem of selecting among heterogeneous on-device IDS models while accounting for detector utility, switching, latency, temperature, and energy. FSSQL-R constructs a calibrated feasible set and applies Q-learning only within that set. We examine whether learning adds practical value beyond the shield, whether the controller is viable in online execution, how a simple threshold controller compares, and how latency, thermal behavior, and measured energy trade off across devices and operating conditions.

The results show that shielding is useful as a device-specific feasibility layer, but Q-learning is not universally superior. On the Pi 3B+, Safe-Greedy outperforms FSSQL-R under a stationary binding regime; on the Pi 4B, FSSQL-R consistently improves F1 and reward over Safe-Greedy, while the no-training Threshold controller has the highest mean F1 in all three evaluated loads. Physical energy differences are small and not consistently significant. We therefore make four contributions:

- 1) We formulate the guard as a device-calibrated feasible-set mechanism whose empirical margins constrain action selection without implying an unconditional end-to-end safety guarantee.
- 2) We compare FSSQL-R with a conventional threshold controller and other baselines using matched-seed analysis, and examine learning stability over 6000 training windows, 1280 states, and 5120 state-action pairs.
- 3) We evaluate real online closed-loop execution on two Raspberry Pi configurations, including controller and switching overhead, independent device-specific guard calibration, and background-load robustness.
- 4) We measure DC input-side power and energy directly with a POWER-Z KM003C and evaluate all 82,332 official UNSW-NB15 test rows once using fixed preprocessing and detector thresholds, keeping deployment-workload results separate from independent detector generalization.

II. RELATED WORK AND POSITIONING

A. Sustainable scheduling and adaptive edge inference

Recent work in sustainable computing increasingly treats resource management as an online control problem rather than a static configuration task. In *IEEE Transactions on Sustainable Computing* (TSUSC), Wang *et al.* use deep reinforcement learning (DRL) for runtime GPU frequency capping [11]; Ko *et al.* schedule energy-harvesting IoT sleep states while preserving reconstruction quality [12]; Gao *et al.* jointly optimize latency and energy for in-network task scheduling [13]; and Li *et al.* learn DVFS policies for multitask edge workloads under deadlines [14]. Recent TSUSC work also highlights adaptation in IoT anomaly detection under concept drift [15]. These studies motivate dynamic resource-aware control, but their actions are frequency, sleep, task-placement, or adaptation decisions rather than selection among heterogeneous on-device IDS models.

Closely related edge-inference work in *IEEE Transactions on Mobile Computing* (TMC) reinforces the need to account for workload variation and the cost of the controller itself. Tan *et al.* formulate workload-adaptive edge inference request scheduling as an MDP and use RL to select among model-serving choices [16]; Zhang *et al.* co-optimize DVFS and edge-cloud offloading for latency/energy efficiency [17]; and thermal-aware scheduling has been studied explicitly for mobile DNN execution [3]. Our problem differs in two respects: model selection is constrained by a separately calibrated per-window feasible set, and the revised evaluation measures complete online controller latency and physical input energy on the target device rather than relying only on a modeled cost.

B. Lightweight and online intrusion detection

Lightweight IDS research spans classical one-class methods, extreme learning machines, compact deep models, and TinyML-style deployment [18]–[23]. More recent high-end security literature has emphasized adaptation as well as compact execution. Nakip and Gelenbe propose an online self-supervised IDS that adapts without conventional offline relabeling [24], while Yang *et al.* address lightweight dynamic open-set intrusion detection for IIoT [25]. These works focus on improving or updating the detector. The practical problem here is complementary: the detectors are frozen before formal evaluation, and the runtime controller decides which detector to execute under latency, thermal, and resource constraints.

C. Safe RL and shielding

Safe RL covers expectation-constrained optimization, Lyapunov methods, and action shielding [5]–[7], [26]. FSSQL-R is not novel because shielding itself is new; rather, it instantiates a calibrated, device-specific feasible-action layer for multi-model IDS scheduling and separates this layer from utility optimization. Malwi *et al.* [10] use a runtime safety shield for self-healing remediation actions in 5G-IoT. Our action semantics and constraints are different: the actions are heterogeneous IDS models, and admissibility is derived from device-specific latency and thermal measurements under explicit operating caps.

D. Positioning against simpler controllers

A key lesson from the revision is that learning must be compared with controllers that do not require training. The baseline set therefore includes Static-Light, Round-Robin, Safe-Greedy, Unshielded-Q, and a conventional threshold controller. Safe-Greedy uses the same calibrated feasible set as FSSQL-R but makes a one-step security-utility choice. Unshielded-Q uses the same state, reward, and Q-learning implementation without feasible-set masking. The threshold controller uses only measured state and fixed development-time thresholds. Earlier energy-aware and multi-objective precursor formulations are not archival publications and are not used to establish novelty. The numeric Q-learning configuration used here is reported explicitly, fixed before formal evaluation, and evaluated against these simpler alternatives.

III. PROBLEM FORMULATION

We consider fixed decision windows of length $\Delta = 100$ ms. An action $a_t \in \mathcal{A} = \{1, \dots, K\}$ selects one of $K = 4$ frozen IDS models. The state is

$$s_t = (z_t, T_t, b_t, q_t, a_{t-1}), \quad (1)$$

where z_t is a lightweight risk/workload descriptor, T_t is CPU temperature, b_t is a normalized energy-state proxy used by the controller reward, q_t is backlog state, and a_{t-1} captures switching history.

The per-window utility is

$$r_t = r_{\text{sec}}(z_t, a_t) - \lambda_E e(a_t) - \lambda_Q \psi(q_t) - \lambda_{\text{scsw}}(a_{t-1}, a_t), \quad (2)$$

where r_{sec} combines the risk estimate with fixed validation TPR/FPR, and the remaining terms penalize compute/energy proxy, backlog, and switching. The deployment constraints are

$$\tau(s_t, a_t) \leq \tau_{\text{max}} = 40 \text{ ms}, \quad T_{t+1}(s_t, a_t) \leq T_{\text{max}} = 82^\circ\text{C}. \quad (3)$$

Let $\hat{\tau}(s, a)$ and $\hat{T}_{t+1}(s, a)$ be device-calibrated one-step predictors. With one-sided residual margins $\epsilon_{\tau, a}$ and ϵ_T , the feasible set is

$$\mathcal{A}_{\text{safe}}(s) = \{a \in \mathcal{A} : \hat{\tau}(s, a) + \epsilon_{\tau, a} \leq \tau_{\text{max}}, \hat{T}_{t+1}(s, a) + \epsilon_T \leq T_{\text{max}}\}. \quad (4)$$

If $\mathcal{A}_{\text{safe}}$ is nonempty, FSSQL-R chooses the maximizing Q action inside this set. Empty-set handling uses the fixed least-violating fallback, although no fallback occurred in the accepted formal online campaigns.

IV. FEASIBLE-SET SHIELDED Q-LEARNING

A. Calibrated guard

The guard is calibrated separately for each hardware configuration. Latency uses per-action/workload/background empirical predictors; thermal prediction uses a fitted one-step model. Predictor fitting, residual-margin estimation, and held-out coverage evaluation are separated. We use a Q0.99 one-sided residual margin with a fixed conservative buffer selected before held-out coverage is evaluated.

TABLE I
RECENT WORK MOST CLOSELY RELATED TO SUSTAINABLE SCHEDULING, EDGE INFERENCE, AND ADAPTIVE IDS.

Work	Venue	Runtime decision	Main evidence/constraint	Relation to this work
Wang <i>et al.</i> [11]	TSUSC 2024	GPU frequency cap	DRL-based runtime power control	Sustainable runtime control; no IDS feasible-set guard
Xu <i>et al.</i> [15]	TSUSC 2024	Drift adaptation	IoT anomaly adaptation across datasets	Adaptive security model rather than detector scheduling
Ko <i>et al.</i> [12]	TSUSC 2025	IoT sleep scheduling	Energy harvesting and outage-aware DRL	Energy-aware IoT control; different action and constraint semantics
Gao <i>et al.</i> [13]	TSUSC 2025	In-network task scheduling	Joint latency/energy optimization	Related multi-objective scheduling, primarily network/compute placement
Li <i>et al.</i> [14]	TSUSC 2025	DVFS selection	Deadline-aware edge execution	Device adaptation through frequency, not IDS model selection
Tan <i>et al.</i> [16]	TMC 2024	Edge requests/models inference	Workload-adaptive RL scheduling	Closest adaptive inference framing; no calibrated IDS shield
Zhang <i>et al.</i> [17]	TMC 2024	DVFS + offloading	Physical heterogeneous edge evaluation	Latency/energy control through DVFS/offloading
Nakip and Gelenbe [24]; Yang <i>et al.</i> [25]	TIFS 2024–25	Online/open-set IDS adaptation	Dynamic IDS learning and lightweight execution	Improves detector adaptation; our focus is constrained runtime selection among frozen detectors
This work	TSUSC	IDS model selection	Device-calibrated latency/thermal guard, complete online timing, direct DC input energy	Higher-level complement to DVFS/power control; no consistent energy-saving claim

B. Masked Q-learning

For a nonterminal transition (s_t, a_t, r_t, s_{t+1}) , the update is

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha_t [r_t + \gamma \max_{a' \in \mathcal{A}_{\text{safe}}(s_{t+1})} Q(s_{t+1}, a') - Q(s_t, a_t)]. \quad (5)$$

The implementation has 1280 discretized states and 5120 state-action pairs. We use $\gamma = 0.95$, $\alpha = \min\{0.5, (1 + N(s, a))^{-0.6}\}$, and linearly decay ϵ from 1.0 to 0.05 by window 4500, retaining 0.05 thereafter. A FIFO replay buffer of 512 transitions performs one online update plus four uniformly sampled replay updates per training window. Formal evaluation uses $\epsilon = 0$ and disables Q/replay updates.

C. Conditional safety statement

The safety result is conditional rather than absolute.

Proposition 1 (Conditional one-step admissibility). *If the true latency and next-temperature errors are bounded by the calibrated guard margins for a state-action pair, every action admitted by (4) satisfies the modeled one-step caps in (3).*

Proof. For an admitted action, $\hat{\tau} + \epsilon_{\tau, a} \leq \tau_{\max}$ and $\hat{T}_{t+1} + \epsilon_T \leq T_{\max}$. If the corresponding positive prediction errors do not exceed the margins, the true quantities are bounded by the same cap. $\square$ $\square$

The empirical residual distribution can violate this assumption. Moreover, complete end-to-end latency includes controller/runtime effects beyond the modeled detector/service component. We therefore report held-out coverage and realized online violations explicitly rather than describing the calibration as an unconditional guarantee.

V. EXPERIMENTAL METHOD

A. Data and model pool

We use UNSW-NB15 [27]. The official training file is split into group-safe development partitions for model training, validation, and scheduler development. All detector-quality

TABLE II
CONSOLIDATED EXPERIMENTAL PARAMETERS.

Parameter	Value
Decision window	100 ms
Latency cap $\tau_{\max}$	40 ms
Thermal cap $T_{\max}$	82°C
Actions	4 IDS models
Q states / state-actions	1280 / 5120
Discount γ	0.95
Replay capacity	512
Pi 3B+ binding matrix	6 policies $\times$ 10 seeds $\times$ 600 windows
Pi 4B online matrix	4 policies $\times$ 3 loads $\times$ 5 seeds $\times$ 600 windows
Pi 4B loads	$n = 16$, B0/B1/B2 background CPU
Energy meter	POWER-Z KM003C, direct HID
Energy boundary	DC input side of Pi 4B power path

terms used by the scheduler reward are fixed from the validation partition; official-test statistics do not enter model training, guard calibration, reward construction, Q-learning, or policy selection. The official 82,332-row test set is sealed throughout calibration, scheduler training, hardware experiments, and statistical analysis, then opened once for the final detector-generalization evaluation. The fixed action pool is FISVDD, LUCID, TinyDL, and OI-SVDD+AS-ELM.

B. Hardware and calibration

We evaluate a Raspberry Pi 3 Model B+ and a Raspberry Pi 4 Model B with 8 GB RAM. The Pi 3B+ is operated with a heatsink and continuously active fan inside an open-top case, with a stabilized wall-powered supply path; the Pi 4B uses active cooling. Accepted formal runs use the performance governor and reject current throttling/undervoltage events or frequency violations. The two devices are calibrated independently; Pi 3B+ predictor/margin artifacts are never reused on the Pi 4B.

Calibration uses 72 action/workload/background cells (4 actions $\times$ 6 batch sizes $\times$ 3 background conditions) with separate fit, margin, and held-out coverage splits, each containing 2880 windows. Table III reports the accepted held-out coverage.

TABLE III
HELD-OUT EMPIRICAL GUARD COVERAGE (NO POST-COVERAGE TUNING).

Device	Latency	Thermal	Joint
Pi 3B+	0.999653	1.000000	0.999653
Pi 4B 8GB	1.000000	0.998958	0.998958

C. Evidence separation and runtime accounting

The evaluation separates four evidence roles that can otherwise be conflated in adaptive systems. First, detector fitting and scheduler development use only the official UNSW-NB15 training file. Ground-truth labels associated with the frozen scheduler-development workload are held as a scoring sidecar: they are unavailable to preprocessing, risk estimation, state construction, guard evaluation, reward construction, Q-learning, and action selection, and are joined only after the selected detector has produced a prediction. Second, guard calibration uses disjoint fit, margin, and coverage windows on each device, and coverage observations never modify the accepted predictor or margin. Third, formal online policy comparisons use frozen device-specific guard/Q artifacts and fixed run orders. Finally, the official test file is opened only after all development-side choices and hardware analyses are frozen.

We distinguish action/service latency from complete online latency. The guard predicts the service component associated with executing a candidate detector under the calibrated workload/background state. The formal worker separately measures preprocessing, telemetry, state construction, guard/admissible-set evaluation, policy selection, detector inference, switching, and complete end-to-end (E2E) elapsed time using the Pi monotonic clock. This separation is necessary because a detector can satisfy the calibrated service bound while operating-system or controller jitter creates an E2E tail event. The controller is therefore treated as part of the deployment cost rather than assumed negligible.

For Pi 4B energy measurements, KM003C acquisition runs on the Mac orchestrator and does not enter the Pi controller critical path. Meter samples and run-boundary acknowledgements share the Mac monotonic clock domain; Pi monotonic timestamps are used only for internal latency accounting. Raw meter traces include pre/post roll and are integrated over the acknowledged run boundaries. Technical acquisition failures are retained as failed attempts rather than silently overwritten; only the predefined technically valid replacement attempt enters the accepted scientific cell.

D. Baselines and online execution

Safe-Greedy uses the same $\mathcal{A}_{\text{safe}}$ as FSSQL-R but maximizes current security utility without temporal learning. Unshielded-Q removes feasible-set masking. Static-Light fixes the light action. Round-Robin cycles through the pool. The added no-training Threshold controller uses development-calibrated temperature/backlog/risk thresholds: high temperature or backlog selects the light action; sufficiently cool/low-backlog high-risk state selects the highest-validation-utility action; otherwise it retains the previous action.

On the Pi 3B+, a stationary binding-resource matrix compares six policies over 10 matched seeds. The accepted guard has $|\mathcal{A}_{\text{safe}}| = 3$ in every window and excludes LUCID. On the Pi 4B, we run 60 complete online experiments: FSSQL-R, Safe-Greedy, Threshold, and Static-Light under B0, B1, and B2 background load, with five matched seeds and 600 windows per run. Every window records preprocessing, telemetry, state construction, guard/admissible-set construction, policy selection, Q lookup, detector inference, switching, total controller time, complete end-to-end latency, temperature, frequency, fault flags, action, and post-action confusion-matrix contributions.

E. Physical energy measurement

A POWER-Z KM003C measures the Pi 4B DC input path at approximately 10 samples/s. Each formal run includes at least 5 s pre-roll and 5 s post-roll. Energy is computed by trapezoidal integration of measured power using actual Mac monotonic timestamps over the run ACK boundaries. Two initial runs with invalid meter binding were preserved as technical failures and repeated under the predefined technical invalidation rule; the accepted set contains 60 complete scientific cells and 36,000 windows. Primary energy results are absolute input energy without idle subtraction.

F. Statistics

The statistical unit is the matched run/seed, never a pooled window. Pi 3B+ paired comparisons use 10 matched seeds; Pi 4B uses five matched seeds per condition. We report raw paired values, mean/SD, bootstrap 95% CIs for mean differences, paired tests, and effect sizes. Because $n = 5$ limits exact nonparametric power, we report both paired t and exact Wilcoxon results for the Pi 4B comparison rather than treating one test as definitive.

VI. RESULTS

A. RQ1: Does calibrated shielding create a meaningful feasible set?

The Pi 3B+ held-out joint coverage is 2879/2880 (0.999653); the single uncovered point is retained. Under the binding-resource experiment, the guard excludes LUCID and maintains exactly three admissible actions in all 36,000 windows. Thus, on this constrained device the shield actively changes the action set. In contrast, all Pi 4B online windows retain all four actions ($|\mathcal{A}_{\text{safe}}| = 4$). This cross-device result shows why calibration is hardware-specific: the same 40-ms operating cap is binding on the Pi 3B+ workload but nonbinding on the stronger Pi 4B for the evaluated online conditions.

One Pi 3B+ Threshold window exceeds the 40-ms complete E2E cap (52.522 ms) despite the selected TinyDL action having a guarded predicted latency of 5.147 ms with 24.297 ms headroom. Detector inference is only 0.446 ms; the event is dominated by a 50.414-ms controller/runtime outlier. This retained event motivates the claim boundary: the guard calibrates modeled action/service latency and thermal feasibility, while complete pipeline timing must be measured empirically.

TABLE IV
Pi 3B+ BINDING-RESOURCE RESULTS, MEAN OVER 10 MATCHED SEEDS.

Policy	F1	Reward	E2E ms	Switch cost	Viol.
FSSQL-R	0.8523	0.5434	8.3149	5.348	0
Safe-Greedy	0.8708	0.5839	7.9775	0	0
Threshold	0.8704	0.5834	7.9704	0.012	1
Unshielded-Q	0.8523	0.5489	10.1141	4.438	0
Round-Robin	0.8148	0.4801	10.6571	11.980	0
Static-Light	0.7203	0.3389	8.2550	0	0

B. RQ2: Does Q-learning add value over Safe-Greedy?

Table IV reports the Pi 3B+ binding regime. In this regime, FSSQL-R does not improve on Safe-Greedy. Its mean F1 and reward are lower, mean E2E latency is higher, and switching is substantially greater; both methods have zero realized violations. Exact paired sign-flip tests give $p = 0.00195$ for F1, reward, and switching cost, and $p = 0.00586$ for mean E2E latency. The mean controller-overhead difference is not significant ($p = 0.23242$).

A predefined stateful-transition extension was stopped before policy evaluation because measured action service rates could not simultaneously establish the required backlog accumulation under B2/B1 and recovery under B0 without inventing a new arrival-rate rule. We therefore do not use it as evidence for or against Q-learning.

On Pi 4B, the result differs, but the comparison requires an important qualification. Table V shows higher F1 and cumulative reward for FSSQL-R than Safe-Greedy under all three background loads. However, Safe-Greedy collapses to FISVDD in all 15 formal runs and consequently has $\text{FPR} = 1.0000$ and $\text{TN} = 0$ in every run. We retain it because it is the direct same-guard, one-step reference, but it represents an extreme operating point rather than a competitive detector selector. The paired F1 mean differences are +0.1209 (B0), +0.1172 (B1), and +0.1141 (B2), with bootstrap 95% CIs that remain positive; paired t -test p values are 0.011–0.015 and exact Wilcoxon $p = 0.0625$ in each condition. We therefore describe this as a consistent paired advantage over Safe-Greedy, not as overall superiority.

These results also answer the request for a simple controller. Threshold is the strongest detector-selection baseline by mean F1 in all three Pi 4B conditions, and it also has much lower mean FPR (0.0726/0.0581/0.0490 for B0/B1/B2) than FSSQL-R (0.2936/0.3060/0.3109). The FSSQL-R mean, however, masks a reproducible seed-specific failure mode: seed 5102 selects FISVDD in 582/600, 592/600, and 588/600 windows under B0/B1/B2, yielding FPR 0.9144/0.9780/0.9476 and F1 0.8285/0.8245/0.7967. The other four seeds have mean FPR 0.1384/0.1380/0.1517, still above Threshold. Table V therefore reports FPR as median [range], and the supplement gives all five seed-level values. For the paired FSSQL-R-minus-Threshold FPR differences, the bootstrap 95% CIs are [+0.0249,+0.5384], [+0.0323,+0.5928], and [+0.0581,+0.5896], respectively; the corresponding F1-difference intervals cross zero. Given only five pairs, the paired t and Wilcoxon tests for FPR do not cross 0.05, so we interpret the positive bootstrap intervals as directional evidence rather than a universal significance

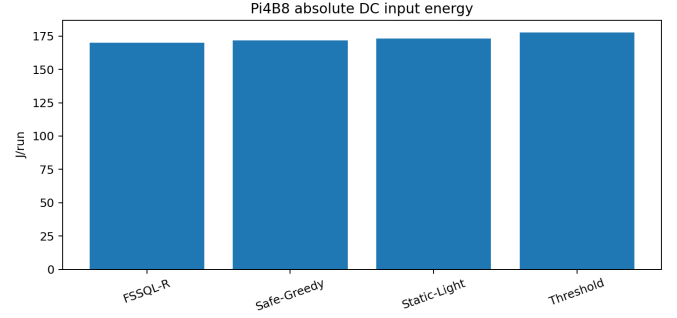


Fig. 1. Absolute Pi 4B DC input energy measured by POWER-Z KM003C. Bars aggregate the accepted formal online runs; absolute input energy is reported without idle subtraction.

claim. Safe-Greedy is retained because it isolates the one-step same-guard decision rule, but its $\text{FPR}=1.0000/\text{TN}=0$ behavior makes it a degenerate detector-selection reference in this Pi 4B experiment. FSSQL-R therefore should not be interpreted as an overall winner.

C. Learning stability and complexity

At 600 training windows, FSSQL-R has mean rolling reward 0.4837, utility 0.5759, and visits 29.7 states/96.3 state-action pairs. At 6000 windows, these become 0.5260, 0.6108, and 34.9/118.9. The greedy-policy change rate falls from 0.006875 to 0.000156. Unshielded-Q shows a similar reduction. Because state support remains sparse and the absolute TD error remains nonzero, this evidence supports late-budget behavioral stability over visited states, not mathematical convergence over the complete 1280-state space.

D. RQ3: What is the measured deployment cost?

Controller work is material relative to detector inference. Across all Pi 4B conditions, mean controller time is 3.919 ms for FSSQL-R and 3.885 ms for Safe-Greedy, whereas mean selected-detector inference is about 0.415 and 0.426 ms, respectively. Thus, telemetry/state/guard/policy logic must be included when assessing deployment cost. FSSQL-R also switches more often (mean 26.8 switches/run across the three conditions) than Safe-Greedy (zero under the fixed utility rule).

Physical power results are shown in Fig. 1. The FSSQL-R minus Safe-Greedy paired mean energy differences are -0.471 J (B0), -3.427 J (B1), and -1.210 J (B2). These differences do not provide consistent statistical evidence for a physical energy advantage: B0/B2 CIs span zero, and the B1 exact Wilcoxon result is $p = 0.0625$. We therefore report energy as a measured operating trade-off rather than a demonstrated energy saving.

E. RQ4: Independent detector generalization

After model, preprocessing, threshold, and scheduling choices were fixed, the official UNSW-NB15 test was evaluated once on all 82,332 rows. Table VI reports independent detector generalization. TinyDL provides the best F1 (0.8686) and the best AUROC/PR-AUC. FISVDD is retained as an adverse

TABLE V

PI 4B 8GB ONLINE RESULTS ACROSS FIVE MATCHED SEEDS PER CONDITION. F1, REWARD, LATENCY, CONTROLLER TIME AND ENERGY ARE MEAN $\pm$ SD; FPR IS MEDIAN [MIN,MAX] BECAUSE ITS SEED DISTRIBUTION IS STRONGLY NON-GAUSSIAN.

Load	Policy	F1	FPR med. [range]	Cum. reward	P99 E2E (ms)	Controller (ms)	Energy (J)	Viol.
B0	FSSQL-R	0.9235 $\pm$ 0.0534	0.1917 [0.0674,0.9144]	333.8 $\pm$ 52.2	5.386 $\pm$ 0.151	3.909 $\pm$ 0.164	173.93 $\pm$ 12.46	0
B0	Safe-Greedy	0.8025 $\pm$ 0.0117	1.0000 [1.0000,1.0000]	204.4 $\pm$ 19.5	5.063 $\pm$ 0.279	3.872 $\pm$ 0.023	174.40 $\pm$ 9.42	0
B0	Static-Light	0.9332 $\pm$ 0.0120	0.2190 [0.1865,0.2767]	346.2 $\pm$ 17.9	5.727 $\pm$ 0.936	4.114 $\pm$ 0.170	175.47 $\pm$ 19.98	0
B0	Threshold	0.9491 $\pm$ 0.0117	0.0714 [0.0622,0.0874]	362.0 $\pm$ 16.5	5.436 $\pm$ 1.118	3.834 $\pm$ 0.189	183.66 $\pm$ 6.24	0
B1	FSSQL-R	0.9271 $\pm$ 0.0580	0.1869 [0.0444,0.9780]	341.6 $\pm$ 58.3	5.021 $\pm$ 0.328	3.897 $\pm$ 0.156	164.50 $\pm$ 0.58	0
B1	Safe-Greedy	0.8099 $\pm$ 0.0117	1.0000 [1.0000,1.0000]	216.8 $\pm$ 19.9	5.150 $\pm$ 0.542	3.879 $\pm$ 0.066	167.93 $\pm$ 3.85	0
B1	Static-Light	0.9387 $\pm$ 0.0065	0.2278 [0.1832,0.2602]	356.2 $\pm$ 9.5	5.481 $\pm$ 0.223	4.075 $\pm$ 0.019	168.80 $\pm$ 5.08	0
B1	Threshold	0.9533 $\pm$ 0.0106	0.0444 [0.0354,0.1173]	371.0 $\pm$ 15.1	7.927 $\pm$ 5.707	4.258 $\pm$ 1.067	181.38 $\pm$ 10.36	1
B2	FSSQL-R	0.9222 $\pm$ 0.0711	0.1882 [0.0378,0.9476]	336.0 $\pm$ 83.5	5.352 $\pm$ 0.576	3.951 $\pm$ 0.119	171.32 $\pm$ 8.16	0
B2	Safe-Greedy	0.8081 $\pm$ 0.0199	1.0000 [1.0000,1.0000]	214.0 $\pm$ 33.8	5.318 $\pm$ 0.573	3.905 $\pm$ 0.093	172.53 $\pm$ 8.11	0
B2	Static-Light	0.9374 $\pm$ 0.0071	0.2270 [0.1832,0.2381]	354.2 $\pm$ 21.1	8.919 $\pm$ 7.298	4.643 $\pm$ 1.193	175.17 $\pm$ 21.45	1
B2	Threshold	0.9627 $\pm$ 0.0040	0.0471 [0.0378,0.0647]	377.0 $\pm$ 13.6	4.913 $\pm$ 0.283	3.747 $\pm$ 0.026	168.24 $\pm$ 4.56	0

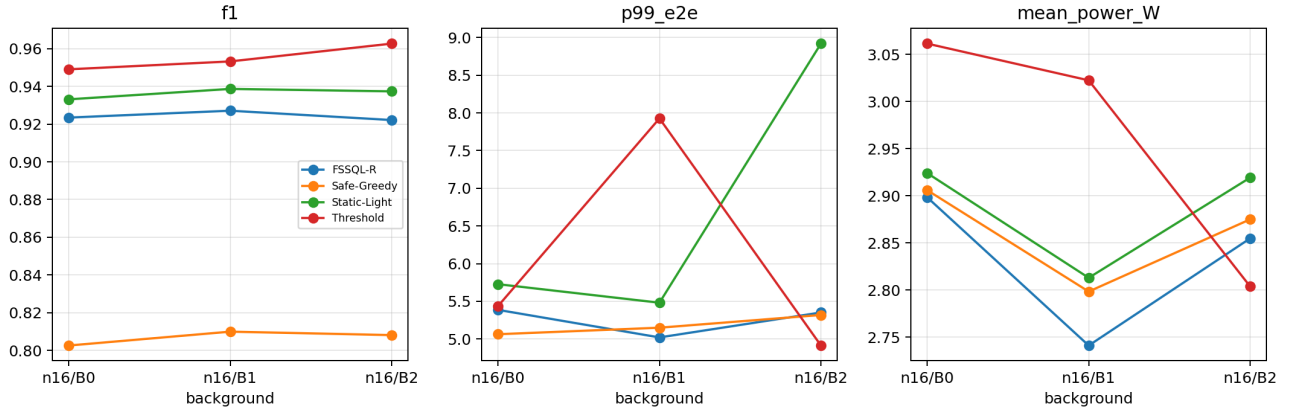


Fig. 2. Pi 4B robustness across B0/B1/B2 background CPU load. F1 is comparatively stable; controller/E2E tails show occasional operating-system/runtime variability, while average DC input power changes modestly.

TABLE VI

OFFICIAL UNSW-NB15 TEST WITH FIXED THRESHOLDS (82,332 ROWS).

Detector	F1	Precision	Recall	FPR
FISVDD	0.7102	0.5506	1.0000	1.0000
LUCID	0.8240	0.7020	0.9973	0.5186
TinyDL	0.8686	0.7802	0.9797	0.3382
OI-SVDD+AS-ELM	0.8521	0.7437	0.9975	0.4211

result: at its frozen threshold it predicts all test rows positive, yielding recall 1.0 but FPR 1.0; its AUROC is only 0.5351, indicating near-random ranking ability rather than threshold misalignment alone. By contrast, TinyDL and OI-SVDD+AS-ELM achieve AUROC 0.9704 and 0.9513, respectively. These test results are not mixed with the Pi 4B scheduler-development online F1 values.

VII. DISCUSSION

A. What the shield establishes

The strongest repeatable conclusion is the calibrated feasibility mechanism. On the Pi 3B+ binding workload, the guard changes the action set by excluding LUCID; on the Pi 4B, the same operating cap is nonbinding under the evaluated online loads. The shield is therefore not universally “necessary for

safety.” Rather, it provides a device-calibrated mechanism for preventing an adaptive policy from selecting actions that are predicted to exceed the calibrated operating envelope when that envelope becomes binding.

B. When is Q -learning worth its complexity?

The two devices do not support a universal answer. In the stationary Pi 3B+ binding regime, learning adds switching cost and reduces F1/reward relative to Safe-Greedy. On Pi 4B, FSSQL-R consistently improves F1 and reward over Safe-Greedy, but Threshold has the highest mean F1 and lowest mean FPR in all three loads, while Static-Light also remains strong. The Pi 4B seed-level results additionally expose a reproducible failure mode: one of five FSSQL-R training seeds (5102) produces a stable FISVDD-dominated evaluation policy in all three loads and yields near-all-positive FPR, whereas the other four seeds remain far lower. With only five seeds this 1/5 observation is not a population failure-rate estimate, but it shows why learning-based schedulers should report per-seed policy behavior and action support rather than only mean performance. A predefined queue-state extension could not establish the required accumulation/recovery dynamics and was stopped before policy runs. Consequently, we do not claim that long-horizon RL is always required; the evidence indicates that its value is operating-regime dependent and seed-sensitive.

C. Sustainability interpretation

We directly measure DC input energy rather than relying on a normalized proxy. This closes an important deployment-evidence gap but changes the claim: FSSQL-R does not demonstrate a consistent physical energy saving. Sustainability is therefore interpreted as measured operation under latency/thermal constraints with explicit energy accounting, not as a guaranteed reduction in joules.

D. Relation to recent sustainable edge control

The recent TSUSC literature shows several successful uses of learning for energy- or resource-aware scheduling, but it also illustrates that the control variable and measurement boundary matter. DRLCap controls GPU frequency at runtime [11]; RASS schedules energy-harvesting IoT sleep states [12]; latency/energy-aware in-network scheduling jointly allocates communication and computation resources [13]; and DRL-based DVFS adapts processor frequency for multitask edge systems [14]. These mechanisms directly manipulate a resource with a strong first-order effect on power or service capacity. FSSQL-R instead chooses among already trained IDS models while leaving the hardware governor fixed during formal evaluation. It is therefore plausible for FSSQL-R to change F1 and reward without producing a large or statistically stable change in whole-device input energy. The direct KM003C result is consequently important not because it confirms an energy-saving hypothesis, but because it prevents a controller-internal energy proxy from being interpreted as a physical joule reduction.

The edge-inference literature makes a related point from a performance perspective. Workload-adaptive scheduling [16], DVFS/offloading co-optimization [17], and thermal-aware mobile inference [3] all exploit platform state to change a serving configuration. Our cross-device result adds a complementary observation: the same nominal 40-ms cap can be binding on a Pi 3B+ and nonbinding on a Pi 4B, so the practical value of a shield depends on the calibrated platform and workload rather than only on the policy design. This is also why a device-specific guard is preferable to transferring margins across hardware.

E. Implications for deployable IDS

Recent TIFS work demonstrates that online adaptation and open-set handling can improve the robustness of IoT/IIoT intrusion detection [24], [25]. Our results identify a different deployment bottleneck: when several competent detectors are already available, the scheduling layer and its telemetry/decision path can become a nontrivial part of total latency. On Pi 4B the mean controller cost is an order of magnitude larger than selected-detector inference for some actions, and the retained Pi 3B+ tail event is dominated by controller/runtime delay rather than classifier execution. A lightweight IDS scheduler should therefore be evaluated as a complete online system rather than by detector latency alone.

The strong Threshold and Static-Light results also have a practical implication. Learning should be retained only

where the operating regime provides enough variation for the additional state/action complexity to pay for itself. The Pi 3B+ stationary binding experiment favors Safe-Greedy, while Pi 4B favors FSSQL-R over Safe-Greedy but not over every simple baseline. This suggests a deployment hierarchy: first calibrate a feasible set, then compare a low-overhead deterministic controller with a learned controller under matched online conditions, and adopt Q-learning only if its incremental detection/utility gain justifies switching and controller cost. Such a hierarchy is consistent with the broader sustainable-computing objective of avoiding unnecessary computation rather than treating learning complexity as intrinsically beneficial.

VIII. LIMITATIONS AND THREATS TO VALIDITY

First, both hardware platforms are Raspberry Pi-class devices; MCU applicability remains prospective. Second, guard coverage is empirical and device/configuration dependent. The 0.999653 and 0.998958 joint coverage values are strong held-out evidence but not mathematical guarantees against all runtime tails. Third, complete E2E latency includes controller and operating-system effects that are not fully represented by the action-level guard; the retained Pi 3B+ controller outlier illustrates this boundary. Fourth, the Pi 4B online F1 values are measured on a fixed labeled scheduler-development workload and are intended for matched deployment comparison, not independent generalization. The official test results in Table VI provide the independent detector-generalization evidence. Fifth, only five matched Pi 4B seeds are available per condition, limiting exact nonparametric power. The repeated seed-5102 FPR collapse is therefore reported as an observed seed-specific failure mode, not as an estimate of its population probability. Finally, the original experimental artifacts were unavailable. The detector, preprocessing, calibration, and scheduling artifacts used in this revision were therefore reconstructed from the submitted method definitions and frozen before evaluation; they are not claimed to be byte-identical historical artifacts, and the revised empirical tables replace rather than numerically reproduce the earlier replay tables. The Q-learning configuration reported here was likewise fixed before the new experiments and is documented explicitly. The conclusions apply to this configuration.

IX. CONCLUSION

This work studies multi-model IDS scheduling under explicit latency and thermal operating limits. FSSQL-R separates calibrated feasible-set construction from masked Q-learning, allowing safety enforcement and long-horizon utility optimization to be examined independently. The experiments clarify the strength and scope of the conclusions. Independent calibration gives joint coverage of 0.999653 on Pi 3B+ and 0.998958 on Pi 4B. The shield actively restricts actions on the constrained Pi 3B+ but is nonbinding on Pi 4B under the tested online loads. Q-learning is not universally superior: Safe-Greedy outperforms FSSQL-R on the Pi 3B+ stationary binding regime, while FSSQL-R consistently improves F1 and reward over Safe-Greedy on Pi 4B. A conventional threshold controller remains a strong baseline. Direct KM003C

measurements provide physical DC input-energy evidence but do not establish a consistent FSSQL-R energy advantage. The resulting contribution is therefore a deployment-grounded account of calibrated feasible-set scheduling, controller cost, and the conditions under which additional learning complexity is or is not justified.

REFERENCES

- [1] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, “MLPerf tiny benchmark,” *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [2] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Damos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, “Mlperf inference benchmark,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.
- [3] T. Tan and G. Cao, “Thermal-aware scheduling for deep learning on mobile devices with npu,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 10 706–10 719, 2024.
- [4] E. Altman, *Constrained Markov Decision Processes*, ser. Stochastic Modeling Series. Taylor & Francis, 1999.
- [5] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 22–31.
- [6] Y. Chow, O. Nachum, E. Duenez-Guzman, and M. Ghavamzadeh, “A lyapunov-based approach to safe reinforcement learning,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.
- [7] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, “Safe reinforcement learning via shielding,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, pp. 2669–2678, Apr. 2018.
- [8] S. Carr, N. Jansen, S. Junges, and U. Topcu, “Safe reinforcement learning via shielding under partial observability,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, pp. 14 748–14 756, Jun. 2023.
- [9] B. Könighofer, R. Bloem, N. Jansen, S. Junges, and S. Pranger, “Shields for safe reinforcement learning,” *Commun. ACM*, vol. 68, no. 11, pp. 80–90, Oct. 2025.
- [10] W. A. Malwi, F. Asiri, N. Alturki, N. Alnazzawi, D. Kasimatis, and N. Pitropakis, “Safety-aware reinforcement learning for self-healing intrusion detection in 5g-enabled iot networks,” *Computers, Materials & Continua*, vol. 87, no. 2, p. 88, 2026.
- [11] Y. Wang, M. Hao, H. He, W. Zhang, Q. Tang, X. Sun, and Z. Wang, “DRLCap: Runtime gpu frequency capping with deep reinforcement learning,” *IEEE Transactions on Sustainable Computing*, vol. 9, no. 5, pp. 712–726, 2024.
- [12] H. Ko, H. Choi, and S. Pack, “Restoration-aware sleep scheduling framework in energy harvesting internet of things: A deep reinforcement learning approach,” *IEEE Transactions on Sustainable Computing*, vol. 10, no. 1, pp. 190–198, 2025.
- [13] J. Gao, F. Zhou, M. Dong, L. Feng, P. Yu, K. Ota, and X. Qiu, “Optimal latency and energy-aware task scheduling in in-network computing paradigm: A deep reinforcement learning approach,” *IEEE Transactions on Sustainable Computing*, vol. 10, no. 4, pp. 690–704, 2025.
- [14] X. Li, T. Zhou, H. Wang, and M. Lin, “Energy-efficient computation with DVFS using deep reinforcement learning for multi-task systems in edge computing,” *IEEE Transactions on Sustainable Computing*, vol. 10, no. 6, pp. 1116–1127, 2025.
- [15] L. Xu, Z. Han, D. Zhao, X. Li, F. Yu, and C. Chen, “Addressing concept drift in IoT anomaly detection: Drift detection, interpretation, and adaptation,” *IEEE Transactions on Sustainable Computing*, vol. 9, no. 6, pp. 913–924, 2024.
- [16] X. Tan, H. Li, X. Xie, L. Guo, N. Ansari, X. Huang, L. Wang, Z. Xu, and Y. Liu, “Reinforcement learning based online request scheduling framework for workload-adaptive edge deep learning inference,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 12, pp. 13 222–13 239, 2024.
- [17] Z. Zhang, Y. Zhao, H. Li, C. Lin, and J. Liu, “DVFO: Learning-based DVFS for energy-efficient edge-cloud collaborative inference,” *IEEE Transactions on Mobile Computing*, vol. 23, no. 10, pp. 9042–9059, 2024.
- [18] D. M. Tax and R. P. Duin, “Support vector data description,” *Machine Learning*, vol. 54, no. 1, pp. 45–66, Dec. 2004.
- [19] G.-B. Huang, Q.-Y. Zhu, and C.-K. Siew, “Extreme learning machine: Theory and applications,” *Neurocomputing*, vol. 70, no. 1, pp. 489–501, 2006, neural Networks.
- [20] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del Rincón, and D. Siracusa, “Lucid: A practical, lightweight deep learning solution for ddos attack detection,” *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020.
- [21] P. Fusco, G. P. Rimoli, and M. Ficco, “TinyIDS – an IoT intrusion detection system by tiny machine learning,” in *Computational Science and Its Applications – ICCSA 2024 Workshops*. Cham: Springer Nature Switzerland, 2024, pp. 71–82.
- [22] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O’Reilly Media, 2019.
- [23] J. Lin, W.-M. Chen, Y. Lin, j. cohn, C. Gan, and S. Han, “MCUNet: Tiny deep learning on IoT devices,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33. Curran Associates, Inc., 2020, pp. 11 711–11 722.
- [24] M. Nakip and E. Gelenbe, “Online self-supervised deep learning for intrusion detection systems,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5668–5683, 2024.
- [25] X. Yang, F. Tong, F. Jiang, and G. Cheng, “A lightweight and dynamic open-set intrusion detection for industrial internet of things,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 2930–2943, 2025.
- [26] J. García and F. Fernández, “A comprehensive survey on safe reinforcement learning,” *Journal of Machine Learning Research*, vol. 16, no. 42, pp. 1437–1480, 2015.
- [27] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *2015 Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6.

Nuonan Ouyang Nuonan Ouyang is a Ph.D. candidate at James Cook University, Australia. His research focuses on lightweight edge/IoT cybersecurity, on-device intrusion detection, and deployable learning-based control under resource constraints.

Adrian Shatte Adrian Shatte is a Senior Lecturer in Information Technology at James Cook University, Australia. His research interests include data science, intelligent systems, digital wellbeing, and applied machine learning.

Zhigang Lu Zhigang Lu is a Senior Lecturer with Western Sydney University, Australia. His research interests include privacy, security, robustness, and trustworthy AI/ML systems.

Chao Chen Chao Chen is an Associate Professor at RMIT University, Australia. His research spans AI for cybersecurity, data analytics, and security of AI systems.

Wei Xiang Wei Xiang is a La Trobe Distinguished Professor and Cisco Research Chair of AI and IoT at La Trobe University, Australia. His research spans communications, IoT, and applied AI.